

ECM2433

Ollie Mock Paper 4

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This converted mock paper has been checked so each question is answerable and answers are not given away in the question text.

Answers and marking notes are at the end of this document.

Question 1

(a) For `int main(int argc, char **argv)`: (i) explain `argc` and `argv`, including `argv[0]`. (ii) Write C code to safely print the first argument after the program name or print an error if none was provided. (6 marks)

(b) C function pointers can pass functions as arguments. (i) Write a prototype for `filter_array` taking an integer array, its length, and a function pointer that takes `int` and returns

`int`. (ii) Explain one advantage of passing comparison/filter logic as a function pointer. (6 marks)

(c) Write a C function `int **allocate_2d(int rows, int cols)` that dynamically allocates a 2D integer array using an array of pointers to rows. Handle allocation failure sensibly. (6 marks)

(d) When compiling large C/C++ projects: (i) what is the advantage of compiling source files into object files before linking? (ii) Explain the role of `-Wall` and `-g` in `gcc`. (7 marks)

(Total 25 marks)

Question 2

(a) A C++ program must call a C function `void calculate_stats(double *data, int n)`. (i) What keyword block should be used in C++? (ii) Explain why this is needed using name mangling. (6 marks)

(b) C++ streams manage error state. (i) If `std::cin >> my_integer` is given "Hello", what happens to the stream and later extractions? (ii) How do manipulators such as `std::hex` or

`std::setprecision` give safer formatting than `printf` format strings? (7 marks)

(c) Use `std::count_if` and a lambda to count how many numbers in `std::vector<int>` numbers are strictly greater than 50. (5 marks)

(d) C++ uses try and catch for exceptions. (i) Write `-l0`. (7 marks)

`double divide(double a, double b)` that throws `std::invalid_argument` if `b` is `0.0`. (ii) Give one reason this is preferable to return

(Total 25 marks)

Question 3

(a) Given the Rust enum below, write `fn print_status(s: &Status)` using `match` to print "Waiting", "Arrived", or "Moving through [City]". (5 marks)

```
enum Status {  
    Pending,  
    InTransit(String),  
    Delivered,  
}
```

(b) In Rust mpsc channels: (i) what does mpsc stand for? (ii) If 3 worker threads send data to one main thread, how should tx be handled? (5 marks)

(c) Rust smart pointers: (i) explain Box<T> versus Rc<T>. (ii) Why does the compiler prevent Rc<T> from crossing a thread boundary? (6 marks)

(d) Given let words = vec![String::from("apple"), String::from("cat")]; write a single iterator chain with iter, map, and collect that creates Vec<usize> lengths. (5 marks)

(e) Explain why accessing p1.dest after the struct update below gives a Rust compiler error. (4 marks)

```
let p1 = Package {  
    id: 1,  
    dest: String::from("London"),  
    weight: 2.0,  
};  
let p2 = Package { id: 2, ..p1 };
```

(Total 25 marks)

Question 4

(a) Polymorphism and dynamic dispatch: (i) In C++, how does virtual enable dynamic polymorphism, including the role of the vtable? (ii) In C, how can dynamic dispatch be simulated without classes? (7 marks)

(b) Metaprogramming and generics: (i) explain how a C preprocessor #define macro differs from a C++ template. (ii) Give one safety reason templates or Rust generics are safer than C macros. (7 marks)

(c) Concurrency and thread safety: (i) what is a data race? (ii) Explain how Rust ownership and mutability rules eliminate data races at compile time. (7 marks)

(d) Compare the C standard library with the C++ STL. Give one high-level data structure available in C++ that C normally requires you to implement manually or get from another library. (4 marks)

(Total 25 marks)

Answers and marking notes

Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (6 marks): For `int main(int argc, char **argv)`: (i) explain `argc` and `argv`, including `argv[0]`. (ii) Write C code to safely print the first argument after the program name or print an error if none was provided.

Answer: `argc` is the number of command-line argument strings. `argv` is the array of argument strings. `argv[0]` is normally the program name/path. The first user argument is `argv[1]`, only valid when `argc >= 2`.

```
if (argc < 2) {
    fprintf(stderr, "missing argument\n");
    return 1;
}
printf("%s\n", argv[1]);
```

(b) Question (6 marks): C function pointers can pass functions as arguments. (i) Write a prototype for `filter_array` taking an integer array, its length, and a function pointer that takes `int` and returns `int`. (ii) Explain one advantage of passing comparison/filter logic as a function pointer.

Answer: The callback type is `int (*predicate)(int)`. Passing a function pointer separates the traversal algorithm from the condition, so the same function can be reused with different filtering logic.

```
#include <stddef.h>
void filter_array(const int values[], size_t len, int (*predicate)(int));
```

(c) Question (6 marks): Write a C function `int **allocate_2d(int rows, int cols)` that dynamically allocates a 2D integer array using an array of pointers to rows. Handle allocation failure sensibly.

Answer: Allocate the row-pointer array first, then each row. If a row allocation fails, free the rows already allocated and then free the row-pointer array.

```
#include <stdlib.h>
int **allocate_2d(int rows, int cols) {
    int **data = malloc((size_t)rows * sizeof *data);
    if (data == NULL) {
        return NULL;
    }
    for (int r = 0; r < rows; r++) {
        data[r] = malloc((size_t)cols * sizeof *data[r]);
        if (data[r] == NULL) {
            for (int i = 0; i < r; i++) {
                free(data[i]);
            }
            free(data);
            return NULL;
        }
    }
    return data;
}
```

(d) Question (7 marks): When compiling large C/C++ projects: (i) what is the advantage of compiling source files into object files before linking? (ii) Explain the role of `-Wall` and `-g` in `gcc`.

Answer: Object files allow separate compilation. If one source file changes, only that file must be recompiled, then the executable is relinked with existing object files. `-Wall` enables many useful compiler warnings. `-g` includes debug information so debuggers and tools can report source-level locations.

Question 2

(a) Question (6 marks): A C++ program must call a C function `void calculate_stats(double *data, int n)`. (i) What keyword block should be used in C++? (ii) Explain why this is needed using name mangling.

Answer: Use `extern "C"` around the C function declaration. C++ normally mangles names, while the C object file exports an unmangled C symbol. C linkage makes the C++ compiler emit/use the symbol name the linker can match.

```
extern "C" {
    void calculate_stats(double *data, int n);
}
```

(b) Question (7 marks): C++ streams manage error state. (i) If `std::cin >> my_integer` is given "Hello", what happens to the stream and later extractions? (ii) How do manipulators such as `std::hex` or `std::setprecision` give safer formatting than `printf` format strings?

Answer: The extraction fails, `failbit` is set, and `my_integer` is not assigned a valid parsed `int`. Later extractions fail until the error state is cleared and bad input is removed. C++ stream formatting is type-aware through overloaded operators and manipulators, whereas

`printf` relies on the programmer matching format specifiers to argument types correctly.

(c) Question (5 marks): Use `std::count_if` and a lambda to count how many numbers in `std::vector<int>` numbers are strictly greater than 50.

Answer: `count_if` calls the predicate for each element and counts those where it returns true.

```
int count = std::count_if(numbers.begin(), numbers.end(),
    [](int value) {
        return value > 50;
    });
```

(d) Question (7 marks): C++ uses try and catch for exceptions. (i) Write `double divide(double a, double b)` that throws `std::invalid_argument` if `b` is 0.0. (ii) Give one reason this is preferable to returning -1.0.

Answer: The exception separates error signalling from valid numeric results. Returning -1.0 is ambiguous if -1.0 could be a real division result.

```
#include <stdexcept>
double divide(double a, double b) {
    if (b == 0.0) {
        throw std::invalid_argument("division by zero");
    }
    return a / b;
}
```

Question 3

(a) Question (5 marks): Given the Rust enum below, write `fn print_status(s: &Status)` using `match` to print "Waiting", "Arrived", or "Moving through [City]".

Code from question:

```
enum Status {
    Pending,
    InTransit(String),
    Delivered,
}
```

Answer: Because `s` is a borrowed `Status`, the match patterns borrow the inner city string rather than moving it out.

```
fn print_status(s: &Status) {
    match s {
        Status::Pending => println!("Waiting"),
        Status::Delivered => println!("Arrived"),
        Status::InTransit(city) => println!("Moving through {city}"),
    }
}
```

(b) Question (5 marks): In Rust `mpsc` channels: (i) what does `mpsc` stand for? (ii) If 3 worker threads send data to one main thread, how should `tx` be handled?

Answer: `mpsc` means multiple producer, single consumer. Clone `tx` for each worker, move the clone into that worker's closure, and keep one `rx` in the main thread. All cloned senders send to the same receiver.

(c) Question (6 marks): Rust smart pointers: (i) explain `Box<T>` versus `Rc<T>`. (ii) Why does the compiler prevent `Rc<T>` from crossing a thread boundary?

Answer: `Box<T>` gives single ownership of one heap allocation. `Rc<T>` gives shared ownership in one thread by reference counting. `Rc<T>` is not `Send/Sync` because its reference count updates are not atomic, so using it across threads could race. Use `Arc<T>` for thread-safe shared ownership.

(d) Question (5 marks): Given `let words = vec![String::from("apple"), String::from("cat")];` write a single iterator chain with `iter`, `map`, and `collect` that creates `Vec<usize>` `lengths`.

Answer: `iter` borrows the strings, `map` converts each borrowed string to its length, and `collect` builds the vector.

```
let lengths: Vec<usize> = words
    .iter()
    .map(|word| word.len())
    .collect();
```

(e) Question (4 marks): Explain why accessing `p1.dest` after the struct update below gives a Rust compiler error.

Code from question:

```
let p1 = Package {
    id: 1,
    dest: String::from("London"), weight: 2.0,
};
let p2 = Package { id: 2, ..p1 };
```

Answer: Struct update syntax moves fields from `p1` that are not explicitly replaced. `dest` is a `String`, so it is moved into `p2` because `String` is not `Copy`. After that, `p1.dest` cannot be used. Copy fields such as integers/floats can still be copied.

Question 4

(a) Question (7 marks): Polymorphism and dynamic dispatch: (i) In C++, how does virtual enable dynamic polymorphism, including the role of the vtable? (ii) In C, how can dynamic dispatch be simulated without classes?

Answer: In C++, virtual functions are dispatched through runtime type information such as a vtable, so a base pointer can call the derived override. In C, similar behaviour can be simulated with structs containing function pointers, or by passing function pointers alongside data.

(b) Question (7 marks): Metaprogramming and generics: (i) explain how a C preprocessor #define macro differs from a C++ template. (ii) Give one safety reason templates or Rust generics are safer than C macros.

Answer: A macro is token substitution before compilation and has little type awareness. A template is compiled by the C++ compiler and instantiated for concrete types with type checking. Templates/generics avoid many macro pitfalls such as double evaluation, precedence bugs, and Answer: lack of type checking.

(c) Question (7 marks): Concurrency and thread safety: (i) what is a data race? (ii) Explain how Rust ownership and mutability rules eliminate data races at compile time.

Answer: A data race occurs when multiple threads access the same memory concurrently, at least one access is a write, and there is no proper synchronisation. Rust prevents this in safe code by enforcing either many immutable references or one mutable reference, and by requiring thread-shared mutation to go through safe synchronisation types such as Mutex.

(d) Question (4 marks): Compare the C standard library with the C++ STL. Give one high-level data structure available in C++ that C normally requires you to implement manually or get from another library.

Answer: C's standard library is lower-level and does not provide generic containers like vectors and maps. C++ STL provides generic containers, iterators, and algorithms. Examples include `std::vector`, `std::map`, `std::set`, and `std::unordered_map`.